

Composition and conflicts

Effect algebras decide what overlapping interventions mean

Two interventions on different targets always compose. Two on the same target compose only if their supports are disjoint or the target's algebra says how to combine them. This vignette shows the algebras and the rules.

```
using CategoricalInterventions
```

Undeclared targets reject overlaps

A `TargetSpace` records the algebra of each target. Anything undeclared uses `Reject`: overlapping effects are a conflict, and the report names the pair, the overlap, and an algebra that would resolve it.

```
plain = TargetSpace()
p = @interventions plain begin
    closure = scale(:beta, 0.6; during=10..30)
    masks   = scale(:beta, 0.8; during=20..40)
end
conflicts(p)
```

1-element Vector{Conflict}:

Conflict(Parameter:beta on [20, 30): closure vs masks; effects scale(0.6) and scale(0.8) do

```
try
    compose(Program(p[1]; space=plain), Program(p[2]; space=plain))
catch err
    sprint(showerror, err)
end
```

"Intervention program has 1 conflict(s):\n - Conflict(Parameter:beta on [20, 30): closure v

Multiplicative and set-then-scale

Declaring `Multiplicative()` makes overlapping scalings multiply. Declaring `Affine(Multiplicative())` (the default for rates in models built from `AlgebraicPetri` and `StockFlow`) additionally allows one absolute assignment, which is applied *before* any scalings, regardless of the order in which the atoms were written.

```
space = TargetSpace()
declare!(space, :beta, Affine(Multiplicative())); value_type=Float64
declare!(space, :gamma, Affine(Multiplicative())); value_type=Float64

q = @interventions space begin
    closure = scale(:beta, 0.6; during=10..30)
    masks   = scale(:beta, 0.8; during=20..40)
    baseline = set(:beta, 0.5; during=15..35)
end
[(e.support, round(e.values[:beta]; sigdigits=4)) for e in apply(q, Dict(:beta => 0.3))]
```

```
5-element Vector{Tuple{Span{Int64}, Float64}}:
 ([10, 15], 0.18)
 ([15, 20], 0.3)
 ([20, 30], 0.24)
 ([30, 35], 0.4)
 ([35, 40], 0.24)
```

On `[20, 30]` the value is $0.5 * 0.6 * 0.8 = 0.24$: the assignment first, then both scalings. Reordering the atoms gives the same schedule.

```
q_reordered = Program(reverse(q.atoms); space=space)
[(e.support, round(e.values[:beta]; sigdigits=4)) for e in apply(q_reordered, Dict(:beta => 0.3))]
[(e.support, round(e.values[:beta]; sigdigits=4)) for e in apply(q, Dict(:beta => 0.3))]
```

```
true
```

Two absolute assignments on the same target still conflict, because there is no canonical value.

```
conflicts(compose(q, Program(set(:beta, 0.1; during=25..45, id=:second); space=space); check=
```

```
1-element Vector{Conflict}:
 Conflict(Parameter:beta on [25, 35): baseline vs second; effects set(0.5) and set(0.1) do not
```

Caps

A `Cap()` algebra combines upper bounds by taking the tighter one. Here two policies cap a contact rate; where they overlap the lower cap applies.

```
declare!(space, :contacts, Cap(); value_type=Float64)
caps = @interventions space begin
    venue_limit = cap(:contacts, 6.0; during=0..20)
    curfew       = cap(:contacts, 3.0; during=10..30)
end
[(e.support, e.values[:contacts]) for e in apply(caps, Dict(:contacts => 10.0))]
```

```
3-element Vector{Tuple{Span{Int64}, Float64}}:
 ([0, 10), 6.0)
 ([10, 20), 3.0)
 ([20, 30), 3.0)
```

Ordered policies are a separate class

`LatestWins()` and `HighestPriority()` always resolve an overlap, so they never conflict, but the result depends on the order of the atoms. They are therefore not partial commutative monoids, and the package keeps them in a separate class so that the order-independence theorems are never claimed for them.

```
ordered = TargetSpace()
declare!(ordered, :beta, LatestWins())
a = set(:beta, 0.2; during=0..10, id=:a)
b = set(:beta, 0.1; during=5..15, id=:b)
ab = [(e.support, e.values[:beta]) for e in apply(Program([a, b]; space=ordered), Dict(:beta => 1.0))]
ba = [(e.support, e.values[:beta]) for e in apply(Program([b, a]; space=ordered), Dict(:beta => 1.0))]
ab, ba
```

```
(Tuple{Span{Int64}, Float64}[([0, 5), 0.2), ([5, 10), 0.1), ([10, 15), 0.1)], Tuple{Span{Int64}, Float64}[([5, 10), 0.1), ([10, 15), 0.1)])
```

Composition is associative and refines epochs

Composing programs concatenates their atoms and merges their declaration tables. The empty program is the identity, and the bracketing does not matter.

```
r = Program(scale(:gamma, 2.0; during=0..50, id=:testing); space=space)
compose(compose(q, caps), r) == compose(q, compose(caps, r))
```

true

Adding atoms only ever refines the epoch partition: every epoch of the composite lies inside an epoch of the original or misses all of them.

```
before = [e.support for e in epochs(q)]
after  = [e.support for e in epochs(compose(q, r))]
before, after
```

```
(Span{Int64}[[10, 15), [15, 20), [20, 30), [30, 35), [35, 40)], Span{Int64}[[0, 10), [10, 15), [15, 20), [20, 30), [30, 35), [35, 40)])
```

What the category theory bought

Each algebra is a partial commutative monoid, so the order in which overlapping atoms are folded never matters (`fold_perm`, `apply_comm`), and a program is conflict-free exactly when every pair of co-active atoms combines (`conflictFree_of_pairwise`), which is why the cheap pairwise check in `conflicts` is sound. The set-then-scale behaviour is the Affine construction, whose action law is proved as `affine_action`: absolute first, then the product of the relative effects. Composition of programs is list append (`compose_assoc`, `compose_nil_left`, `compose_nil_right`), and adding atoms refines epochs (`epochs_refine`). Ordered policies are deliberately outside these theorems.